CropCare AI

Crop Disease Detection System

Product Requirements Document

Hackathon Edition · Version 1.0 · March 2026

Document Type	Product Requirements Document (PRD)
Project Name	CropCare AI - Crop Disease Detection System
Version	1.0 (Hackathon Edition)
Date	March 2026
Scope	MVP - 24 to 72 Hour Hackathon Build
Primary Users	Small-scale farmers, Agricultural students, Agro-extension workers
Build Timeline	24-72 hours (Hackathon)
Target Regions	Sub-Saharan Africa, South Asia, Southeast Asia

1. Product Overview

CropCare AI is a web-based application that empowers farmers, agricultural students, and extension workers to diagnose crop diseases instantly by uploading a photograph of an affected leaf. Using AI-powered image recognition, the system returns a probable disease diagnosis, confidence score, and actionable treatment and prevention recommendations — all within seconds and without requiring agricultural expertise.

Core Value Proposition

Democratize agricultural disease diagnosis for the 500M+ smallholder farmers globally who currently lack access to agronomists. Reduce crop losses by enabling early, accurate, and affordable detection.

Problem Solved	Farmers lose 20-40% of crops annually due to undetected or misdiagnosed plant diseases
Solution	AI-powered image diagnosis accessible via any smartphone browser
Key Technology	React (frontend), Python FastAPI (backend), CNN / Plant Disease API (AI model)
Differentiator	Offline-aware design, multilingual support, low-data mode for rural access
Demo Impact	Live upload -> diagnosis -> treatment in under 10 seconds

2. Problem Statement

The Agricultural Crisis

Plant diseases account for annual crop losses of 20-40% globally, with the highest impact in developing regions where smallholder farming is the primary source of food and income. The core challenge is threefold:

- Access Gap: Farmers in rural areas have limited or no access to trained agronomists or plant pathologists
- Speed Gap: By the time a diagnosis is obtained through traditional channels, disease spread may be irreversible
- Cost Gap: Professional agricultural consultations are unaffordable for subsistence farmers earning under $2/day
- Knowledge Gap: Misidentified diseases lead to wrong treatments, wasting resources and worsening crop health

Specific Pain Points

Smallholder Farmers	Cannot identify diseases early; rely on neighbor advice or costly consultants
Agro-Extension Workers	Cover too many farmers; need faster tools to scale their impact
Agricultural Students	Lack practical exposure to diverse disease presentations
Food Security	Delayed or incorrect treatment contributes to regional food insecurity

3. Objectives & Goals

Primary Objectives

- Build a functional MVP that detects common crop diseases from leaf images with >80% accuracy
- Deliver results in under 10 seconds from image upload to diagnosis display
- Provide clear, jargon-free treatment and prevention recommendations
- Ensure accessibility on low-end smartphones with poor internet connectivity
- Demonstrate measurable real-world impact in a hackathon demo context

Hackathon-Specific Goals

- Functional demo deployable and shareable via URL within 72 hours
- Impressive judge experience: fast, visually clean, and clearly impactful
- Scalable architecture: clearly communicable beyond MVP in presentations

Success Criteria (MVP)

Detection Accuracy	Greater than 80% on common diseases using test dataset
Response Time	Less than 10 seconds end-to-end from upload to result
Mobile Compatibility	Works on Android Chrome, iOS Safari, and low-spec devices
Languages	English (MVP); Hausa, Swahili, Hindi as stretch goals
Crop Coverage	At least 5 crops and 10 diseases for demo

4. Target Users & Personas

Persona 1: Amara - The Smallholder Farmer

Age / Location	38 years old, Rural Nigeria (Kano State)
Education	Primary school; basic smartphone literacy
Tech Access	2G/3G mobile data; Android phone (entry-level)
Primary Crop	Maize and tomatoes
Pain Point	Lost 60% of tomato harvest last season to blight she could not identify
Goal	Quickly identify what is wrong with her crops and get guidance she can act on today
Key Need	Simple UI, visual results, voice-friendly language, affordable data usage

Persona 2: Kwame - The Agro-Extension Worker

Age / Location	27 years old, Field officer covering 200 farms in Ghana
Education	BSc Agriculture; moderate tech literacy
Tech Access	4G; Android mid-range smartphone and laptop
Responsibility	Advising 200+ farmers across 5 villages
Pain Point	Cannot physically visit every farm; needs a diagnostic tool he can share
Goal	Use the app to triage farm issues remotely and give evidence-based advice
Key Need	Shareable diagnosis links, confidence scores, professional recommendations

Persona 3: Priya - The Agricultural Student

Age / Location	21 years old, University of Agriculture, Ibadan
Education	Undergrad Agricultural Science; high tech literacy
Tech Access	WiFi campus; modern smartphone and laptop
Use Case	Learning to identify diseases through practical case studies
Pain Point	Textbook descriptions do not match real field presentations
Goal	Use app as a study tool and career exploration platform
Key Need	Detailed disease information, confidence scoring, educational disease profiles

5. User Stories

US-01: Instant Crop Disease Diagnosis

Role	As a smallholder farmer
Goal	I want to upload a photo of my crop leaf and receive an instant disease diagnosis
Benefit	So that I can take immediate action to protect my harvest
Acceptance	Disease name shown within 10 seconds; confidence score displayed; minimum 3 treatment steps shown
Priority	Must Have

US-02: Treatment Recommendations

Role	As a smallholder farmer
Goal	I want to receive specific treatment recommendations in plain language
Benefit	So that I know exactly what to buy or do without needing to consult an expert
Acceptance	Recommendations include: organic remedy, pesticide option, and prevention tip; under 150 words total
Priority	Must Have

US-03: Diagnosis History Log

Role	As an agro-extension worker
Goal	I want to view a history of all diagnoses I have performed
Benefit	So that I can track disease patterns across multiple farms over time
Acceptance	Last 10 diagnoses accessible with date, crop type, disease, and image thumbnail
Priority	Should Have

US-04: Shareable Diagnosis Report

Role	As an agro-extension worker
Goal	I want to share a diagnosis result with a farmer via WhatsApp or SMS
Benefit	So that farmers who cannot use the app directly still benefit from the analysis
Acceptance	Share button generates a link or PDF summary with disease name, photo, and recommendations
Priority	Should Have

US-05: Disease Information Library

Role	As an agricultural student
Goal	I want to browse a library of crop diseases with visual examples and descriptions
Benefit	So that I can learn to identify diseases proactively before they appear on my farm
Acceptance	At least 10 diseases searchable by crop type; each entry includes image, symptoms, and treatment

Priority	Should Have

US-06: Low-Data Mode

Role	As a farmer in a low-connectivity area
Goal	I want the app to compress my image before upload to use minimal data
Benefit	So that I can use the service without incurring high mobile data costs
Acceptance	Image compressed to under 500KB before upload; compression happens client-side
Priority	Should Have

US-07: Multilingual Interface

Role	As a farmer who reads primarily in Hausa or Swahili
Goal	I want the interface and results to be available in my local language
Benefit	So that I can fully understand the recommendations without translation help
Acceptance	Language toggle visible on home screen; at least 2 languages available at demo
Priority	Nice to Have

6. Core Features (Prioritized)

Priority	Feature	Rationale
Must Have	Image Upload Interface	*Core interaction; without it the product does not function*
Must Have	AI Disease Detection Engine	*Central value proposition*
Must Have	Confidence Score Display	*Builds user trust; critical for demo clarity*
Must Have	Treatment & Prevention Recommendations	*Actionable output; primary user need*
Must Have	Mobile-Responsive UI	*Target users primarily access via smartphone*
Must Have	Results Page with Disease Summary	*Clean output for demo impact*
Should Have	Diagnosis History (Session-based)	*Adds value for extension workers*

Priority	Feature	Rationale
Should Have	Shareable Diagnosis Link / PDF Export	*Enables viral reach and demo storytelling*
Should Have	Disease Information Library	*Educational value; differentiates product*
Should Have	Image Compression (Client-side)	*Essential for rural usability*
Should Have	Multiple Crop Type Support (5+ crops)	*Broadens applicability*
Nice to Have	Multilingual Interface (Hausa / Swahili)	*High social impact; post-hackathon feature*
Nice to Have	Offline Detection (PWA / cached model)	*Transformative for no-connectivity zones*
Nice to Have	Geolocation-based Disease Alerts	*Proactive prevention at scale*
Nice to Have	Voice-based Results Readout	*Accessibility for low-literacy users*
Nice to Have	Farm Management Dashboard	*Longitudinal crop health tracking*

7. Functional Requirements

7.1 Image Ingestion

- FR-01: System shall accept JPEG, PNG, and WEBP image formats
- FR-02: System shall support camera capture and file upload
- FR-03: System shall validate image resolution (minimum 224x224 pixels)
- FR-04: System shall compress images client-side to maximum 500KB before transmission
- FR-05: System shall provide preview of uploaded image before submission

7.2 Disease Detection

- FR-06: System shall send image to backend detection API via POST request
- FR-07: System shall return top-3 disease predictions with confidence percentages
- FR-08: System shall return a 'Healthy' classification when no disease is detected
- FR-09: System shall handle unrecognized crop/leaf images with a graceful fallback message
- FR-10: System shall complete detection and return results within 10 seconds

7.3 Results Display

- FR-11: System shall display disease common name and scientific name
- FR-12: System shall display confidence score as both percentage and visual indicator
- FR-13: System shall display at least 3 treatment recommendations
- FR-14: System shall display at least 2 prevention tips

- FR-15: System shall show a reference image of the identified disease for comparison

7.4 History & Sharing

- FR-16: System shall store last 10 diagnoses in browser localStorage
- FR-17: System shall display diagnosis date, crop type, and disease in history log
- FR-18: System shall generate a shareable URL or downloadable PDF for each diagnosis

7.5 Disease Library

- FR-19: System shall provide a searchable library with minimum 10 disease entries
- FR-20: Each disease entry shall include: symptoms, affected crops, treatment, and reference images
- FR-21: Library shall be filterable by crop type

8. Non-Functional Requirements

Performance

- NFR-01: End-to-end response time must be under 10 seconds on 3G network
- NFR-02: Application initial load time must be under 4 seconds on mobile
- NFR-03: API must handle concurrent requests without degradation during demo

Reliability

- NFR-04: System must maintain 99% uptime during the hackathon demo period
- NFR-05: API fallback must handle model service failures with user-friendly error messages
- NFR-06: All user-facing errors must include clear guidance on next steps

Usability

- NFR-07: Core upload-to-result flow must be completable in under 3 user interactions
- NFR-08: Interface must follow WCAG 2.1 Level AA contrast standards
- NFR-09: App must function on devices with 1GB RAM and Android 8.0+

Security

- NFR-10: Uploaded images must be processed in memory only; not permanently stored without consent
- NFR-11: All API communication must occur over HTTPS
- NFR-12: No personally identifiable information shall be collected in MVP

Scalability

- NFR-13: Backend must be horizontally scalable via containerization (Docker)
- NFR-14: AI model inference should be separable from application server for independent scaling

9. System Architecture

Technology Stack

Frontend	React 18, TypeScript, TailwindCSS, Axios
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic
AI / ML	TensorFlow Lite CNN (PlantVillage-trained) or Plant.id API / Google Vision API
Disease Database	JSON flat file (MVP) or PostgreSQL (production)
Image Processing	Pillow (server-side), browser-image-compression (client-side)
Deployment	Vercel (frontend), Render / Railway / Fly.io (backend), Docker
Monitoring	Sentry (error tracking), Uptime Robot (availability)

Component Architecture

- Frontend Layer: React SPA served via CDN; handles image capture, compression, and results rendering
- API Gateway: FastAPI REST API handling image ingestion, preprocessing, and model routing
- AI Inference Layer: TensorFlow Lite model loaded in-process OR external API call to Plant.id / Google Vision
- Disease Knowledge Base: Structured JSON / DB storing disease metadata, treatment protocols, and reference images
- Storage: Browser localStorage for diagnosis history (MVP); S3-compatible object storage for production

Data Flow

1. User captures or selects leaf image on mobile browser
2. React app compresses image client-side (browser-image-compression library)
3. Compressed image sent as multipart/form-data POST to FastAPI /predict endpoint
4. FastAPI preprocesses image: resize to 224x224, normalize pixel values
5. Preprocessed image passed to TF Lite model OR forwarded to Plant.id API

6. Model returns: disease class, confidence scores, top-3 predictions
7. FastAPI enriches result: fetches disease details from knowledge base
8. Enriched JSON response returned to React frontend
9. React renders: diagnosis card, confidence visualization, treatment steps
10. Result saved to browser localStorage; share URL generated

API Endpoints

POST /predict	Accepts image file; returns disease prediction with metadata
GET /diseases	Returns paginated list of all diseases in knowledge base
GET /diseases/{id}	Returns full details for a single disease entry
GET /health	Health check endpoint; returns system status and model version

10. User Flow

Primary Flow: Capture to Diagnosis

11. User opens CropCare AI URL on smartphone browser
12. Landing page displays: prominent 'Scan Your Crop' CTA button
13. User taps button; selects Camera (take photo) or Gallery (upload existing)
14. Image preview screen shows selected leaf photo
15. User confirms image or retakes if needed
16. Loading animation plays ('Analyzing your crop...')
17. Results page renders: disease name, confidence score, severity badge
18. User scrolls to see treatment steps and prevention tips
19. User optionally taps 'Learn More' to view full disease profile
20. User taps 'Share Result' to send via WhatsApp / copy link
21. Result automatically saved to 'My Diagnoses' history

Secondary Flow: Disease Library Browsing

22. User navigates to 'Disease Library' from bottom navigation
23. User browses by crop type or searches by disease name
24. User selects a disease entry to view full profile
25. User reads: symptoms, prevention, treatment, and example images

11. UI/UX Considerations

Design Principles

- Mobile-First: All layouts designed for 360px-480px viewport first; tablet and desktop are enhancements
- 3-Tap Rule: Core task (upload -> result) must complete in 3 taps or fewer
- Visual Hierarchy: Disease name largest; confidence score prominently colored; recommendations easy to scan
- Progressive Disclosure: Show key result first; hide technical details in expandable sections
- Offline Awareness: Clear messaging when connection is lost; graceful degradation

Key UI Elements

- Upload CTA: Full-width green button with camera icon; impossible to miss on landing screen
- Confidence Indicator: Circular gauge (green > 80%, amber 50-80%, red < 50%)
- Severity Badge: Color-coded chips (Mild / Moderate / Severe) at top of result card
- Treatment Cards: Step-numbered cards with icons; organic and chemical options clearly labeled
- Bottom Navigation: 5 items max (Home, Diagnose, Library, History, About)

Accessibility

- Minimum 4.5:1 contrast ratio on all text elements (WCAG AA)
- Touch targets minimum 44x44px on all interactive elements
- Alt text on all diagnostic result images
- Loading states with progress indicators to reduce anxiety on slow connections
- Error messages written in plain language; always include a suggested next action

Color System

Primary Green	#1A5C38 — brand identity; headers and primary actions
Accent Green	#4CAF74 — success states, healthy diagnosis
Warning Amber	#F59E0B — medium confidence or moderate disease severity
Alert Red	#DC2626 — high severity disease; low confidence
Background	#F9FAF8 — warm off-white; reduces eye strain outdoors
Text Primary	#1A1A1A — maximum readability in bright sunlight

12. Success Metrics (KPIs)

Metric	Target	Measurement	Timeline
Detection Accuracy	> 80% on test set	Confusion matrix / precision-recall	At launch
End-to-End Response Time	< 10 seconds on 3G	Browser network trace	At launch
Task Completion Rate	> 90% upload-to-result	Session funnel analytics	Post-demo
Mobile Usability Score	> 85 Lighthouse mobile	Google Lighthouse audit	At launch
User Satisfaction (Demo)	> 4/5 judge rating	Hackathon judge scorecard	Demo day
Diagnoses Performed	> 20 during demo	Backend request log count	Demo day
Share Rate	> 30% of results shared	Share button click events	Post-demo
Crop Coverage	>= 5 crops, 10 diseases	Disease database record count	At launch

13. Assumptions & Constraints

Assumptions

- A pre-trained model (PlantVillage CNN or Plant.id API) can be integrated within 4-6 hours
- Target users have access to a basic smartphone with a functioning camera
- Demo will be conducted on a reliable WiFi network (not rural conditions)
- Disease knowledge base (treatments, descriptions) can be manually curated for 10 diseases within the hackathon
- At least one team member has Python/FastAPI and one has React experience
- Cloud deployment infrastructure (Render, Vercel) is available and free-tier sufficient

Constraints

- Build Time: 24-72 hours maximum; scope must be ruthlessly prioritized
- Team Size: Assumed 2-4 developers; no dedicated designer
- Data Privacy: No real user data collection permitted without consent infrastructure
- Model Accuracy: Pre-trained models may have gaps for hyperlocal African/Asian crop varieties
- Budget: Zero infrastructure spend; free tiers and open-source only
- Language: English only for MVP; translations require additional time investment

14. Risks & Mitigation Strategies

Risk	Likelihood	Impact	Mitigation
AI model accuracy below 80% for local crop varieties	**High**	**High**	Use Plant.id API as primary with TF Lite as fallback; curate test set from local images
External API (Plant.id) down during demo	**Medium**	**High**	Implement mock fallback response with pre-cached demo results for demo resilience
Team unable to integrate model in time	**Medium**	**High**	Start with Plant.id API (30-min integration); build custom model only if time permits
Poor mobile performance on low-end devices	**Medium**	**Medium**	Test on Android Go devices; implement image compression; lazy-load non-critical assets
Backend deployment failure before demo	**Low**	**High**	Deploy 24h before demo; have local Docker fallback on team laptop as backup
Scope creep consuming time on non-MVP features	**High**	**Medium**	Strict feature freeze after Hour 12; backlog all non-Must-Have items
Internet outage at demo venue	**Low**	**High**	Deploy backend locally via ngrok; have demo video prepared as final fallback

15. Future Enhancements (Post-Hackathon)

Phase 2: Scale (Month 1-3)

- Offline-First PWA: Cache model in-browser using TensorFlow.js for zero-connectivity diagnosis
- Multilingual Interface: Hausa, Swahili, Yoruba, Hindi, Bengali via i18n framework
- Custom Model Training: Fine-tune on African and South Asian crop datasets for higher local accuracy
- Farmer Profile & Farm Mapping: Track multiple fields per user with GPS tagging

Phase 3: Intelligence (Month 3-6)

- Disease Spread Alerts: Aggregate anonymized diagnosis data to alert nearby farmers of regional outbreaks
- Seasonal Risk Forecasting: Integrate weather API data with disease likelihood modeling
- Marketplace Integration: Link treatment recommendations to local agrochemical suppliers
- Community Forum: Peer-to-peer advice channel moderated by verified extension workers

Phase 4: Ecosystem (Month 6-12)

- Government & NGO API: Provide disease trend data dashboards to agricultural ministries
- SMS/USSD Interface: Extend access to feature phone users with no smartphone access
- Drone Image Input: Accept aerial field scan inputs for large-scale farm monitoring
- Crop Insurance Integration: Provide certified diagnosis reports for insurance claims processing

🚀 **Scaling Vision**

CropCare AI is designed to evolve from a hackathon tool into a continental agricultural intelligence platform — ultimately reducing smallholder crop loss by 30% across Sub-Saharan Africa and South Asia within 5 years.

Built to Feed the World

CropCare AI · Hackathon Edition · March 2026

Every farmer deserves access to world-class agricultural intelligence.